

DESARROLLO WEB ENTORNO CLIENTE // PRÁCTICA 1 //

JOSÉ MANUEL SANZ // 19.09.26

Desarrollo web en entorno cliente

Unidad 1

NORMAS DE REALIZACIÓN

- La práctica se realizará de forma **individual**.
- La entrega se hará a través del aula virtual mediante un PDF llamado **Práctica1_NombreApellidos.pdf**. Por ejemplo, **Práctica1_AnaPérezLópez.pdf** si tu nombre es Ana Pérez López.
- También es obligatorio que quede registrado **al menos un enlace con algún tutorial**.

ENUNCIADO

El objetivo de esta práctica es investigar sobre los editores más eficientes para JavaScript, así como las diferentes utilidades de los navegadores para depurar y visualizar el código. Para ello, debes llevar a cabo los siguientes pasos y documentarlos para la entrega de la práctica:

1. Antes que nada, asegúrate que tienes al menos tres navegadores, ya sean Microsoft Edge, Mozilla Firefox, Google Chrome, Safari, Opera u otros.
2. Instala un entorno de desarrollo con soporte para JavaScript como Visual Studio Code, Eclipse, Aptana o similares.
3. Busca información sobre los editores de texto plano orientados para desarrollo web, ya sea Notepad++, Atom, Sublime, Brackets u otro. Instala aquel que consideres que se adapta mejor a tus necesidades y forma de programar.
4. Busca un editor online de JavaScript. De nuevo, elige el que mejor se adapta a tus necesidades.

En este punto deberías tener tres utilidades: entorno de desarrollo, editor de texto orientado a la web y editor online. Debes dedicar un mínimo de una página del PDF por cada una de las utilidades para explicar sus características incluyendo imágenes con la interfaz y su funcionamiento.

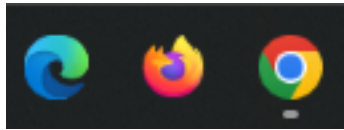
A continuación, completa los apartados que se indican debajo y añade capturas de pantalla de los pasos más relevantes para ejecutar o depurar el código.

5. Abre el código que se adjunta en el enunciado del aula virtual en cada uno de los editores (Ejemplo incrustado.html y Ejemplo externo.html, este último con el archivo script.js). Ejecuta el código mediante las utilidades de cada herramienta (menú para abrir en el navegador, vista previa, botón de ejecutar, es decir, la alternativa correspondiente según el editor).
6. Abre el código de los dos archivos mediante los tres navegadores que tienes en tu equipo. Busca alguna herramienta que te permita acceder al código y visualizarlo.
7. Depura el código estableciendo un punto de ruptura en "console.log(msg)". Ejecuta el código en los tres navegadores depurando paso a paso desde esta sentencia.
8. Busca dónde está la consola y si aparece algún texto tras ejecutar el código.

Para terminar, se ha creado un espacio en el aula virtual para ampliar la respuesta de los diferentes apartados. El objetivo es contribuir con material de utilidad sobre alguno de los entornos empleados, por ejemplo con tutoriales o enlaces de utilidad. Otra alternativa es incluir material sobre el uso del depurador. Es obligatorio añadir al menos un enlace diferente al resto de compañeros. El espacio se encuentra justo debajo del enunciado de la práctica. La práctica no se corregirá si no se añade al menos un enlace.

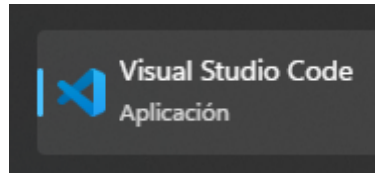
Para comenzar con la práctica, debemos comprobar si tenemos tres navegadores.

En mi caso cuento con Microsoft Edge, Mozilla Firefox y Google Chrome:



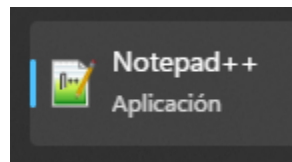
Después debemos instalar un entorno de desarrollo con soporte para JavaScript.

Yo he elegido Visual Studio Code:

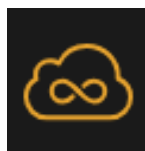


Más tarde necesitaremos un editor de texto plano orientado para desarrollo web.

En mi caso he elegido Notepad++:



Como editor online para JavaScript utilizaremos JSFiddle. En este caso no es necesario instalar nada. Accederemos desde su página web <https://jsfiddle.net/>



¿Por qué Visual Studio Code?

Visual Studio Code es un editor de código fuente desarrollado por Microsoft que permite trabajar con diferentes lenguajes de programación y tecnologías de desarrollo web. En esta práctica se ha seleccionado como **entorno de desarrollo para trabajar con HTML y JavaScript**.

Una de sus principales características es el **soporte específico para JavaScript**. Visual Studio Code incorpora funcionalidades como **autocompletado de código (IntelliSense), navegación por el código, formateado, detección de determinados errores y herramientas de depuración**. Además, dispone de un **sistema de extensiones** que permite ampliar sus funcionalidades según las necesidades del programador.

Otra característica importante es su **integración con proyectos completos**. Los archivos pueden organizarse mediante carpetas y visualizarse desde el explorador situado en la parte izquierda de la interfaz. Esto resulta especialmente útil en el **desarrollo web, donde normalmente se trabaja con varios archivos HTML, CSS y JavaScript**.

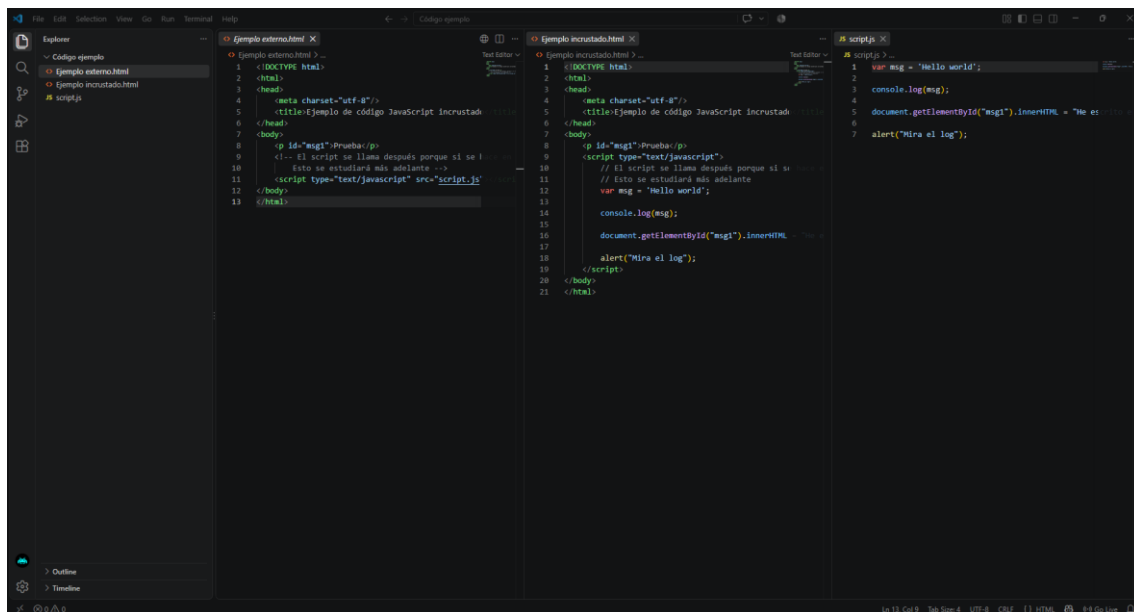
Para esta práctica se ha utilizado **Visual Studio Code para abrir los archivos Ejemplo incrustado.html, Ejemplo externo.html y script.js**. El primero contiene el código JavaScript dentro del propio documento HTML, mientras que el segundo utiliza el **archivo externo script.js**.

Para ejecutar los ejemplos se ha utilizado la **extensión Live Server**, que permite abrir el documento HTML mediante un **servidor local y visualizar inmediatamente el resultado en el navegador**. De esta forma se puede modificar el código y comprobar posteriormente los cambios realizados.

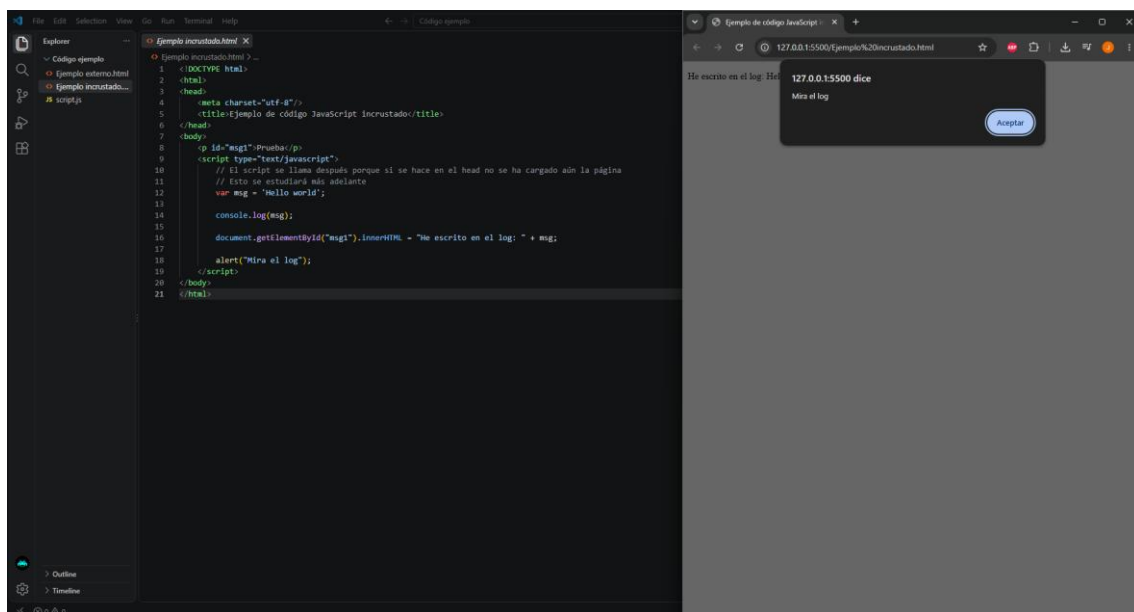
Visual Studio Code también dispone de **herramientas de depuración**. Es posible establecer **puntos de ruptura (breakpoints), detener la ejecución y analizar el comportamiento del código paso a paso**. Estas características son especialmente útiles para **localizar errores y comprobar los valores de las variables durante la ejecución**.

En esta práctica se ha elegido **Visual Studio Code como entorno de desarrollo porque permite trabajar conjuntamente con HTML y JavaScript, organizar los diferentes archivos del proyecto y disponer de herramientas de ejecución y depuración en una misma aplicación**.

En la siguiente captura podemos ver la interfaz de Visual Studio Code con los archivos de la práctica abiertos:



En esta otra captura podemos observar la ejecución de uno de los archivos HTML de la práctica a través de la extensión Live Server:



¿Por qué Notepad++?

Notepad++ es un editor de texto y código fuente gratuito para Windows. Está orientado a la edición de archivos de texto y permite trabajar con numerosos lenguajes de programación y formatos utilizados en el desarrollo web.

Una de sus principales características es el **resaltado de sintaxis**, que permite diferenciar visualmente elementos del código como etiquetas HTML, palabras clave, funciones y cadenas de texto. También dispone de **plegado de código**, lo

que permite ocultar temporalmente determinadas partes de un documento para facilitar su lectura.

Notepad++ permite trabajar con **varios documentos simultáneamente mediante un sistema de pestañas**. Esto resulta útil en esta práctica porque es necesario trabajar con los archivos **Ejemplo incrustado.html**, **Ejemplo externo.html** y **script.js**.

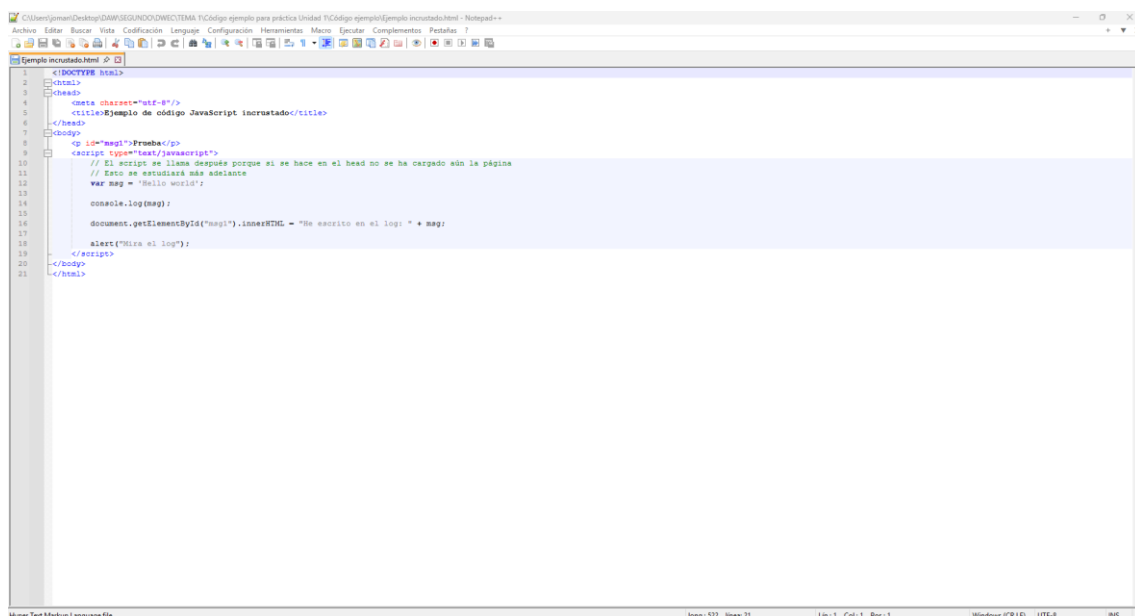
También dispone de herramientas de **búsqueda y sustitución**, **autocompletado** y diferentes opciones para personalizar la interfaz. Además, su funcionalidad puede ampliarse mediante **plugins**.

A diferencia de un entorno de desarrollo completo como Visual Studio Code, Notepad++ está principalmente orientado a la **edición rápida y directa de archivos de texto y código fuente**. En esta práctica se utiliza para **modificar y visualizar el código HTML y JavaScript proporcionado**.

Para trabajar con los ejemplos, se han abierto los archivos HTML y JavaScript directamente desde Notepad++. El editor permite visualizar el código con **resaltado de sintaxis**, facilitando la identificación de las diferentes partes del programa.

Notepad++ ha sido seleccionado como editor de texto orientado al desarrollo web debido a su **sencillez, rapidez y facilidad para trabajar directamente con archivos HTML y JavaScript**.

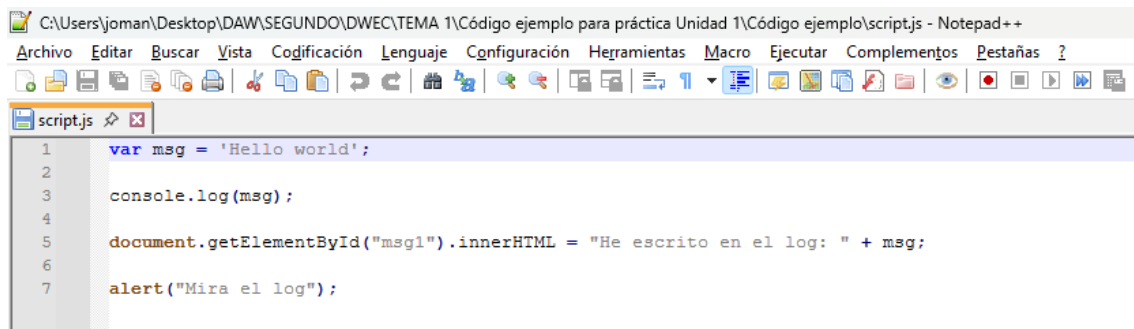
En la siguiente captura vemos la Interfaz de Notepad++ mostrando el archivo HTML de la práctica y su resaltado de sintaxis:



The screenshot shows the Notepad++ interface with a file named 'Ejemplo incrustado.html' open. The code is syntax-highlighted, with HTML tags in blue, attributes in green, and JavaScript code in black. The code includes a DOCTYPE declaration, a head section with a charset and title, and a body section with a div and a script. The script contains comments and a function to log and alert the page content.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8"/>
5 <title>Ejemplo de código JavaScript incrustado</title>
6 </head>
7 <body>
8 <div id="magi">Prueba</div>
9 <script type="text/javascript">
10 // El script se llama después porque si se hace en el head no se ha cargado aún la página
11 // Esto se estudiará más adelante
12 var mag = "Hello world!";
13
14 console.log(mag);
15
16 document.getElementById("magi").innerHTML = "Se escribió en el log: " + mag;
17
18 alert("Mira el log");
19 </script>
20 </body>
21 </html>
```

También podemos ver el archivo js:



```
1  var msg = 'Hello world';
2
3  console.log(msg);
4
5  document.getElementById("msg1").innerHTML = "He escrito en el log: " + msg;
6
7  alert("Mira el log");
```

¿Por qué JSFiddle?

JSFiddle es un entorno de desarrollo online que permite escribir y ejecutar código directamente desde el navegador. No requiere instalar un programa en el ordenador y está orientado principalmente a realizar pruebas y experimentar con código web.

Su interfaz se organiza mediante diferentes paneles. Los principales son **HTML, CSS y JavaScript**, además de un panel de resultado en el que se muestra la ejecución del código. De esta manera, el usuario puede separar la estructura de la página, los estilos y el comportamiento mediante JavaScript.

Para ejecutar el código introducido en los paneles se utiliza el botón Run. Al ejecutarlo, **JSFiddle procesa el código y muestra el resultado en el panel correspondiente.**

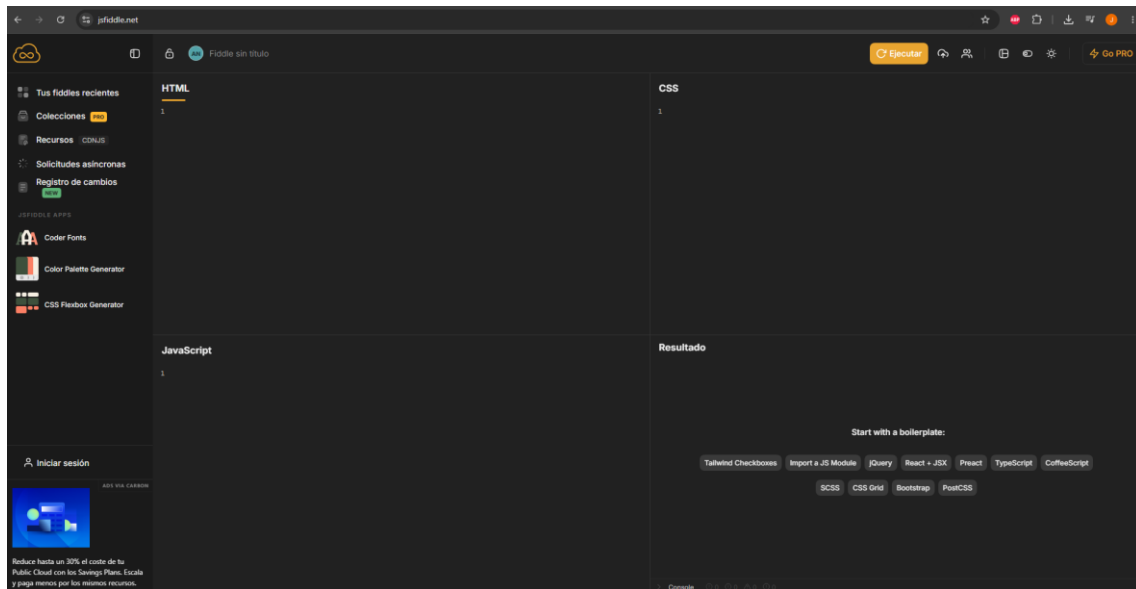
En esta práctica se ha utilizado **JSFiddle para introducir el código HTML y JavaScript del ejemplo proporcionado.** En el panel HTML se ha colocado el elemento que contiene el identificador msg1, mientras que en el panel JavaScript se ha introducido el código encargado de **modificar dicho elemento, escribir un mensaje mediante console.log() y mostrar un cuadro de diálogo.**

Otra característica útil de JSFiddle es la posibilidad de utilizar una **consola JavaScript integrada**, que permite visualizar mensajes generados durante la ejecución del código. Esto permite comprobar, entre otras cosas, el resultado de instrucciones como console.log().

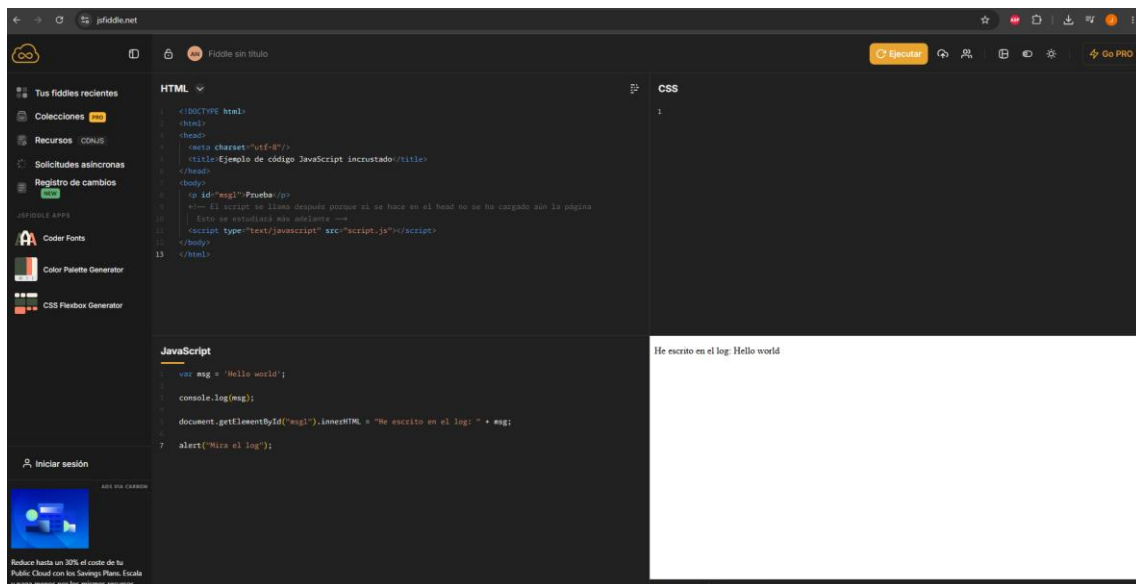
Una de las principales ventajas de JSFiddle es la **rapidez para realizar pruebas.** Al funcionar directamente desde el navegador, **no es necesario crear un proyecto local ni instalar un entorno de desarrollo.** Por este motivo, resulta especialmente útil para experimentar con pequeños fragmentos de **HTML, CSS y JavaScript.**

En esta práctica se ha seleccionado **JSFiddle** como editor online porque permite **comprobar rápidamente el funcionamiento del código JavaScript y visualizar el resultado desde una única interfaz.**

En esta captura podemos ver la interfaz de JSFiddle sin ningún código escrito:



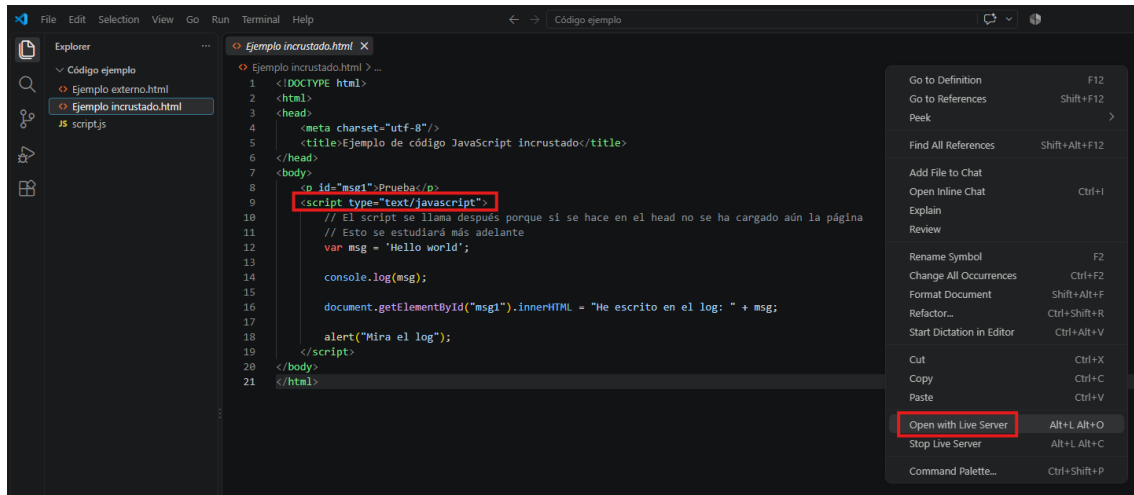
En esta otra captura podemos ver la interfaz con los códigos de los archivos “Ejemplo Externo.html” y “script.js”:



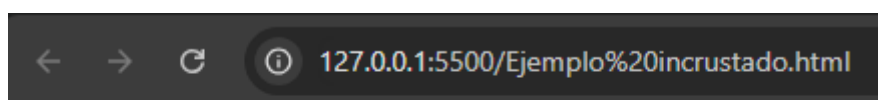
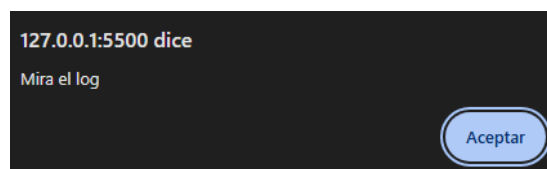
En este punto abriremos en cada uno de los editores el código que se adjunta en el enunciado del Aula Virtual. Primero lo haremos con Visual Studio Code, luego con Notepad++ y por último con JSFiddle.

Visual Studio Code

Probaremos primero “*Ejemplo incrustado.html*”. Para ello, en el Visual Studio Code abriremos la carpeta “*Código Ejemplo*” y después seleccionaremos el archivo. En este archivo el JavaScript está dentro del propio HTML en la línea `<script type="text/javascript">`. Luego lo ejecutaremos con la extensión Live Server. Haremos click derecho y seleccionaremos “*Open with Live Server*”

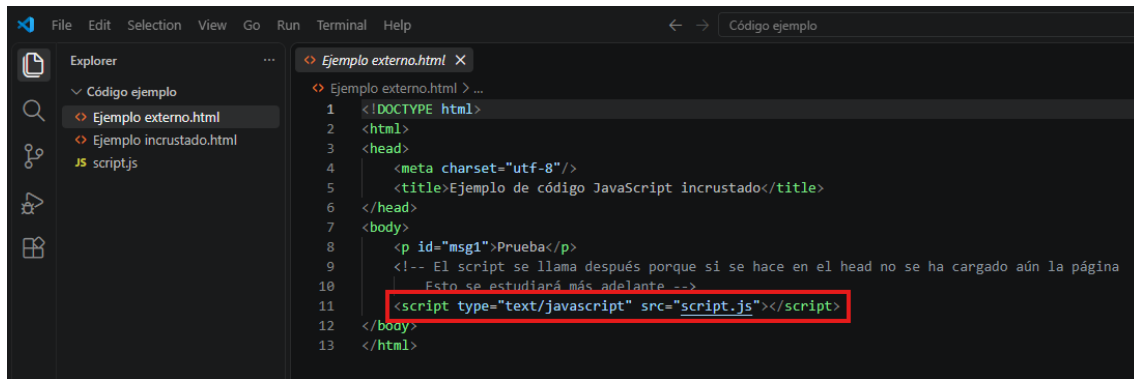


Este es el resultado, después de aceptar la alerta. La alerta salta porque aparece la línea “`alert("Mira el log");`” en el archivo “*script.js*”. Si borramos esa línea, no aparecería dicha alerta.



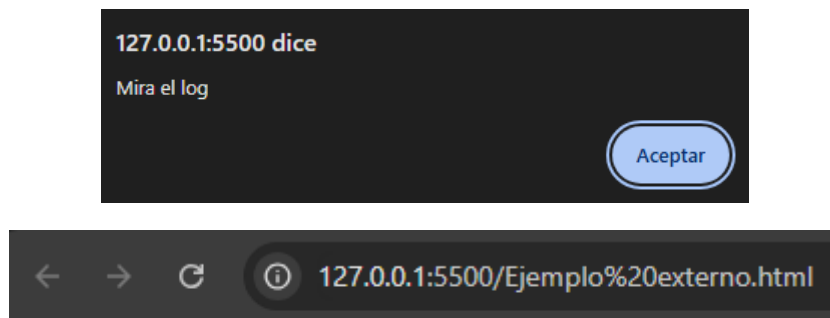
He escrito en el log: Hello world

Ahora repetiremos los mismos pasos con “*Ejemplo externo.html*”. Podemos comprobar que dentro del código se hace referencia al archivo externo “*script.js*”:



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="utf-8"/>
5   <title>Ejemplo de código JavaScript incrustado</title>
6 </head>
7 <body>
8   <p id="msg1">Prueba</p>
9   <!-- El script se llama después porque si se hace en el head no se ha cargado aún la página
10      Esto se estudiará más adelante -->
11   <script type="text/javascript" src="script.js"></script>
12 </body>
13 </html>
```

Aceptamos la alerta igual que antes, y este es el resultado:



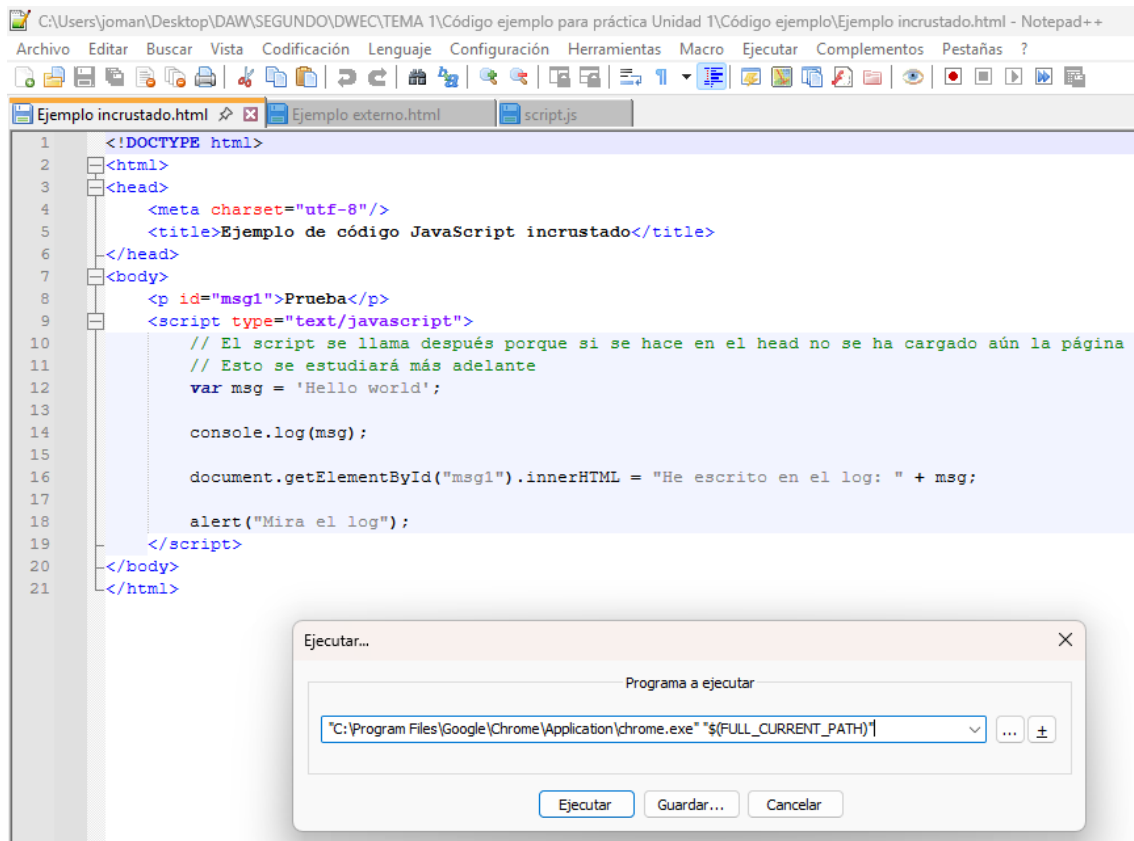
He escrito en el log: Hello world

Notepad++

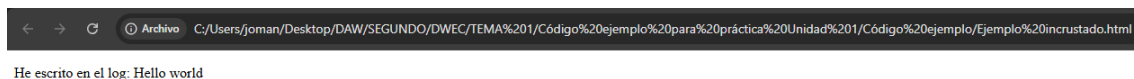
Aquí será un poco distinto a Visual Studio Code. Notepad++ es principalmente un editor de código, no un entorno de ejecución como tal. La parte buena es que permite abrir archivos mediante un navegador utilizando la opción “Ejecutar”.

Primero abriremos el archivo “Ejemplo incrustado.html”. Después, haremos click en “Ejecutar” o pulsaremos “F5”.

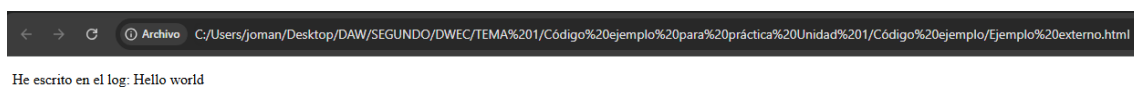
En mi caso, al no estar configurado, me preguntaba qué programa quería ejecutar. Simplemente seleccionaremos la ruta donde tengo instalado Chrome y además “\$(FULL_CURRENT_PATH)” que significa que abra el archivo que está abierto actualmente en Notepad++



Después, pulsaremos “Ejecutar” y este es el resultado:



Ahora abriremos el archivo “Ejemplo externo.html” siguiendo los mismos pasos.
Este es el resultado:



JSFiddle

En este caso hay una pequeña variación. JSFiddle no funciona como una carpeta de archivos html+js tradicional, si no que tiene paneles de HTML, CSS, JavaScript y Result. Por tanto, simplemente copiaremos el código de los archivos en sus respectivos paneles.

Comenzaremos con el archivo “*Ejemplo incrustado.html*”. En el panel HTML sólo copiaremos el código de HTML y en el panel de JavaScript sólo copiaremos el código de JavaScript:

HTML

```
1 <p id="msg1">Prueba</p>
```

JavaScript

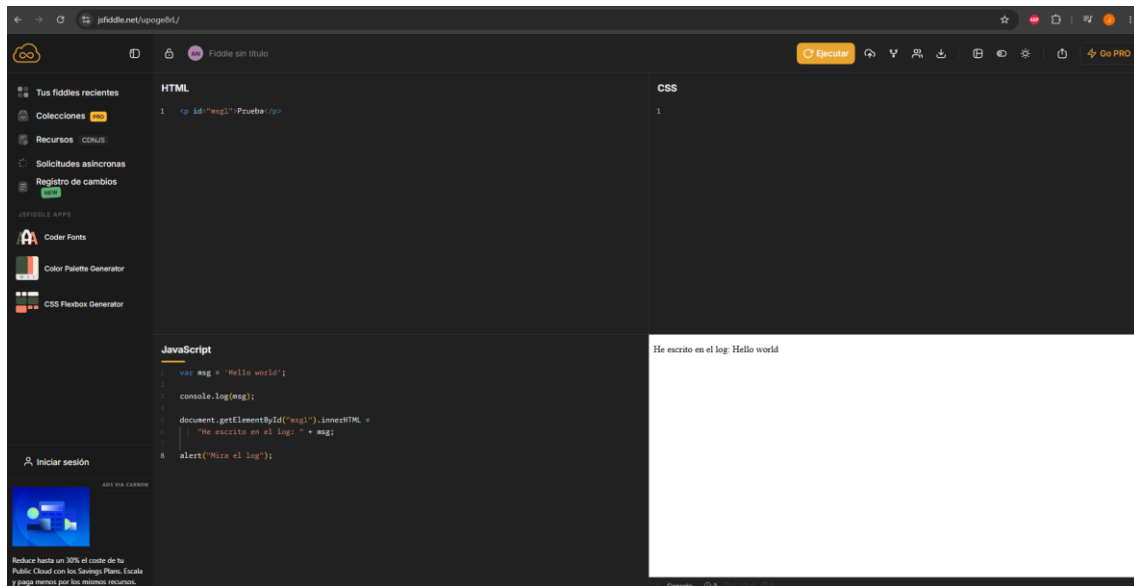
```
1 var msg = 'Hello world';
2
3 console.log(msg);
4
5 document.getElementById("msg1").innerHTML =
6 |   "He escrito en el log: " + msg;
7 |
8 alert("Mira el log");
```

Luego pulsaremos “*Ejecutar*” y este es el resultado:

Una página insertada en fiddle.jsshell.net dice

Mira el log

Aceptar

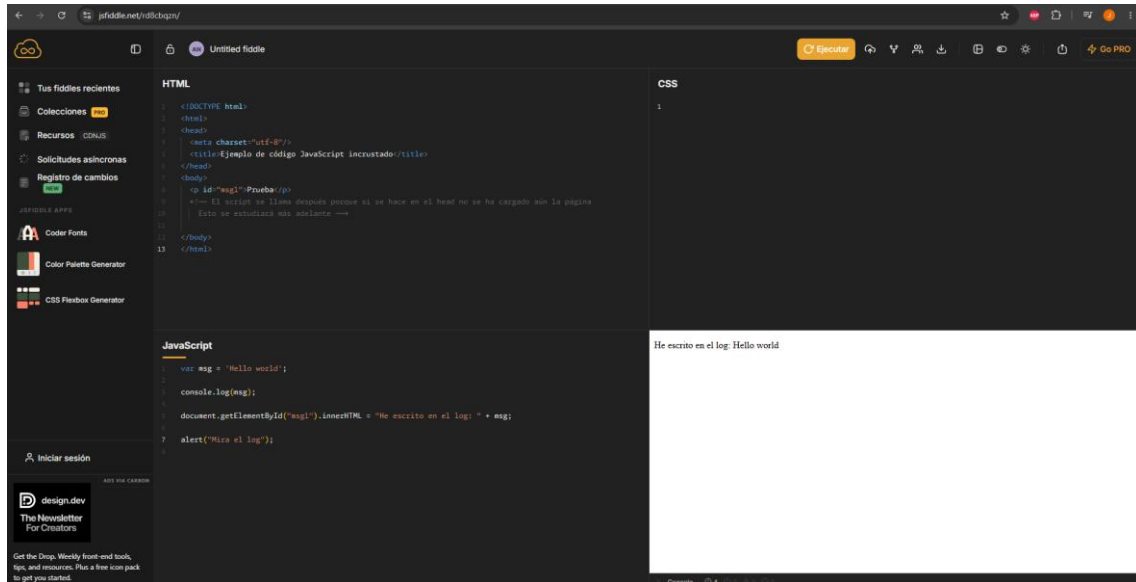
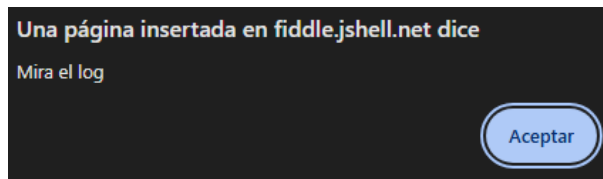


Luego haremos lo mismo con “*Ejemplo externo.html*” y “*script.js*”.

```
HTML
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <meta charset="utf-8"/>
5    <title>Ejemplo de código JavaScript incrustado</title>
6  </head>
7  <body>
8    <p id="msg1">Prueba</p>
9    <!-- El script se llama después porque si se hace en el head no se ha cargado aún la página
10   Esto se estudiará más adelante -->
11  </body>
12 </html>
```

```
JavaScript
1  var msg = 'Hello world';
2
3  console.log(msg);
4
5  document.getElementById("msg1").innerHTML = "He escrito en el log: " + msg;
6
7  alert("Mira el log");
8
```

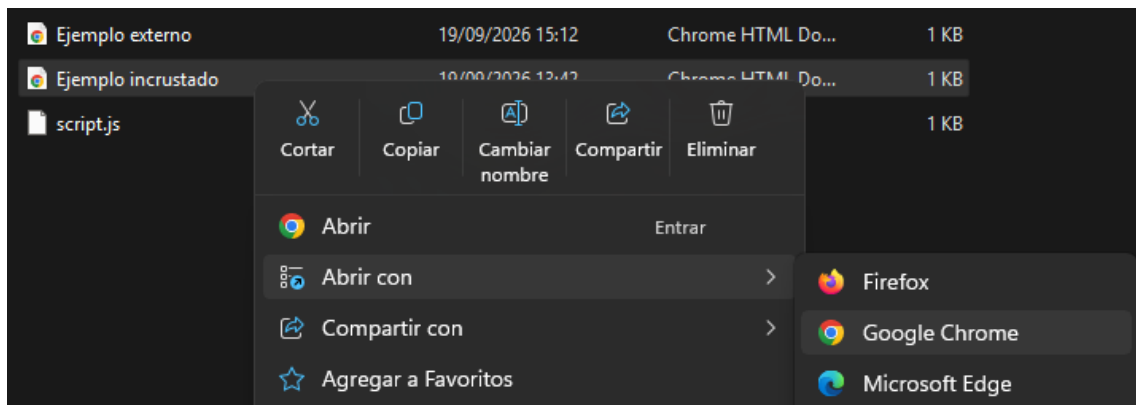
Luego pulsaremos “Ejecutar” y este es el resultado:



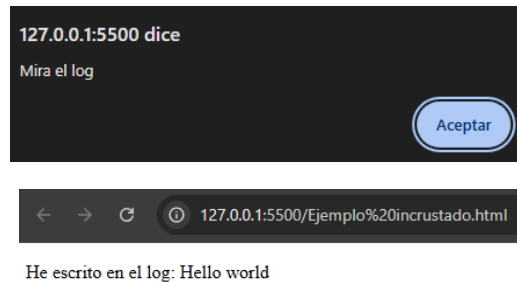
En el anterior punto hemos demostrado que los archivos funcionan en Visual Studio Code, Notepad++ y JSFiddle. En este punto, vamos a comprobar que podemos abrir el código en los tres navegadores

Google Chrome

Comenzaremos con “*Ejemplo incrustado.html*”. Tenemos varias opciones para abrirlo con Chrome. En este caso lo haremos seleccionando el archivo > “Abrir con” > “Google Chrome”



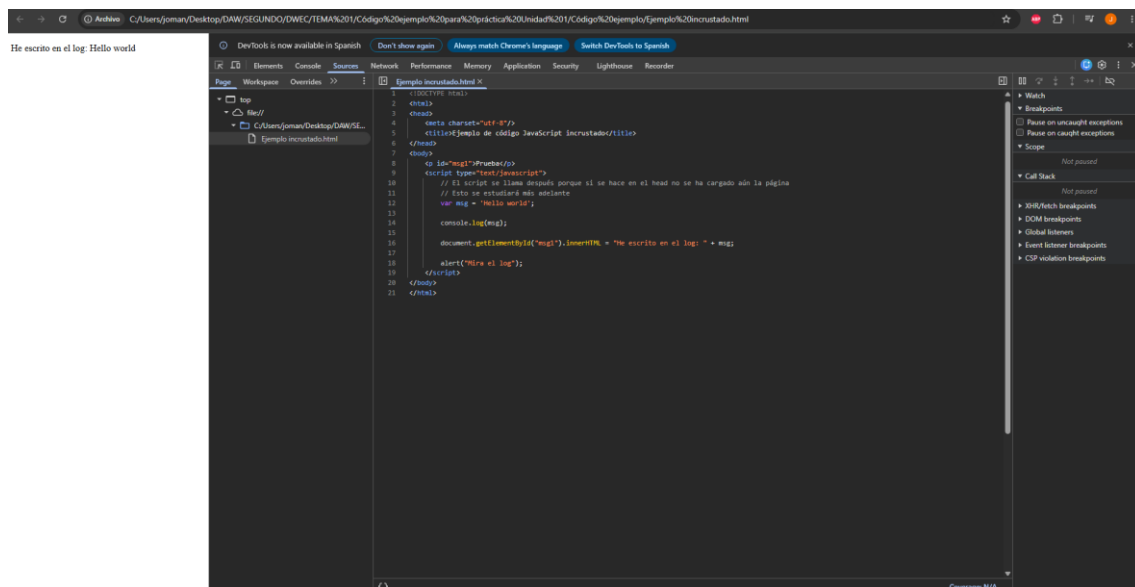
Comprobamos que se muestra lo siguiente:



He escrito en el log: Hello world

Ahora, para abrir las herramientas de desarrollador de Chrome (*DevTools*) también tenemos varias opciones. En este caso hemos pulsado “F12”. Aunque también podemos pulsar “Ctrl + Shift + I” o acceder desde “:> Más herramientas > Herramientas para desarrolladores”.

Una vez dentro, iremos a “Sources” y seleccionaremos “Ejemplo incrustado.html”. Ahí podemos ver el código. Este es el resultado:

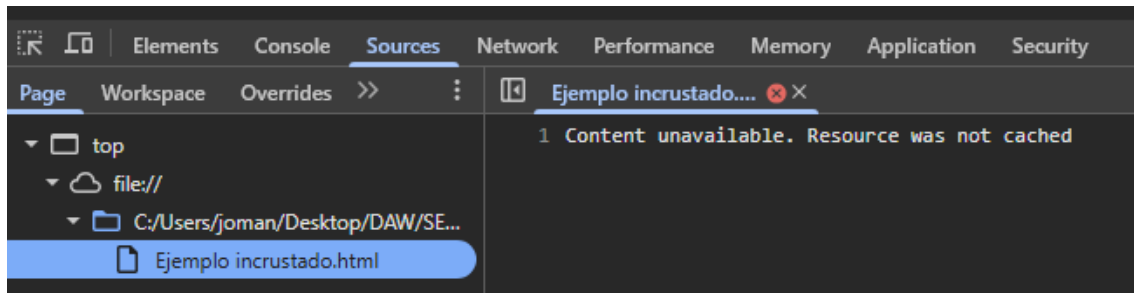


****NOTA****

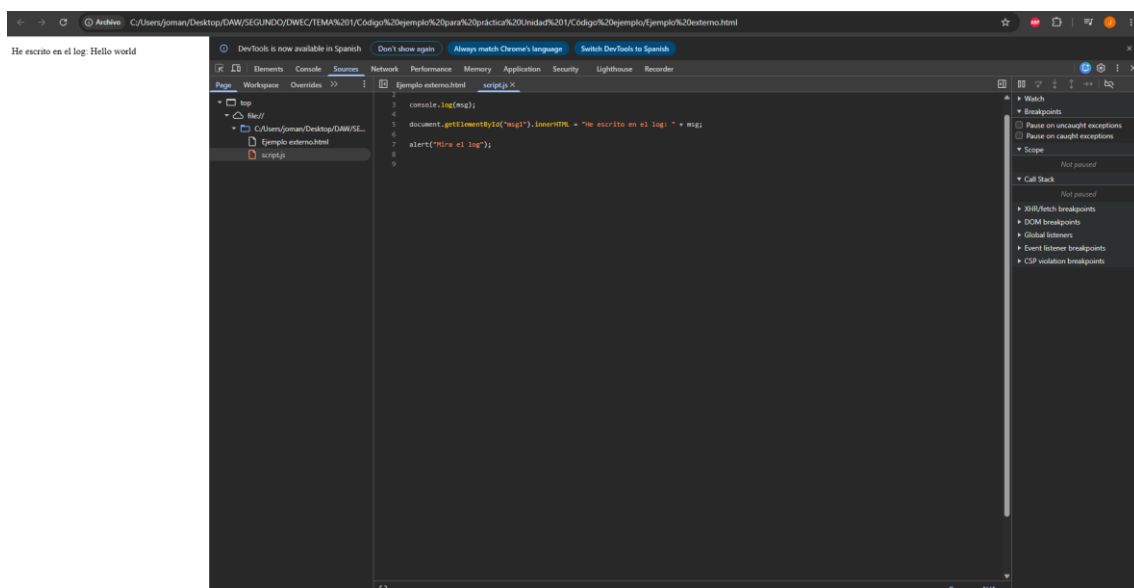
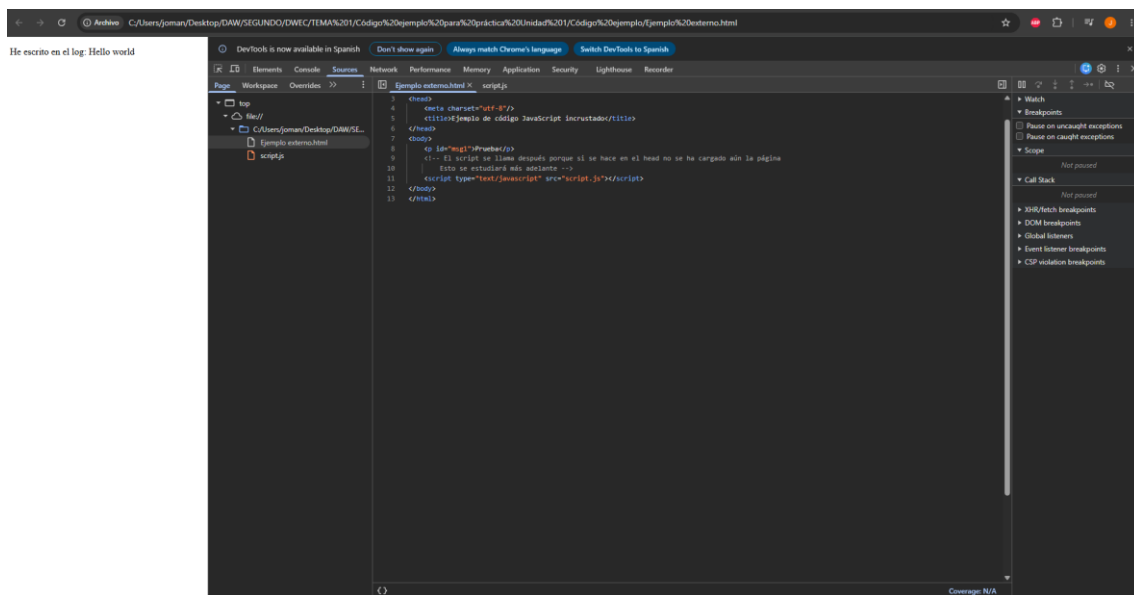
Al principio no me funcionaba. Me aparecía el error “Content unavailable. Resource was not cached” como indico en la captura de abajo.

Esto se debía a que, a veces, si se abre el *DevTools* después de que la página ya haya cargado, los recursos “antiguos” ya no están disponibles.

Conseguí solucionarlo de una forma bastante sencilla, simplemente recargando la página con el *DevTools* abierto. De esta manera ya funcionaba tal y como se indica en la captura anterior de arriba.



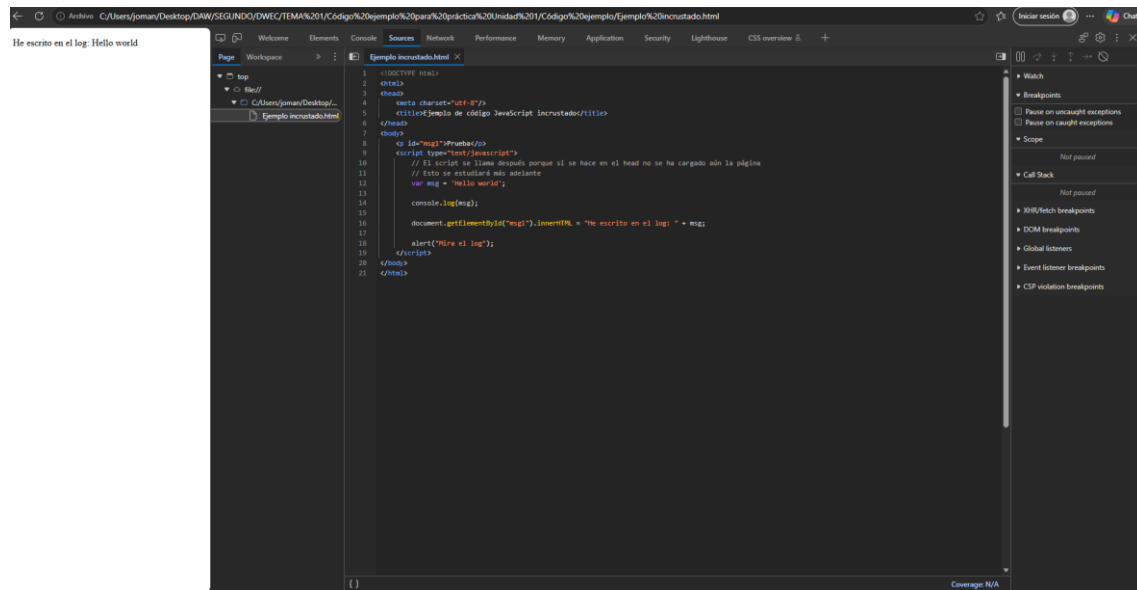
Después haremos lo mismo con “*Ejemplo externo.html*” y “*script.js*”. Estos son los resultados:



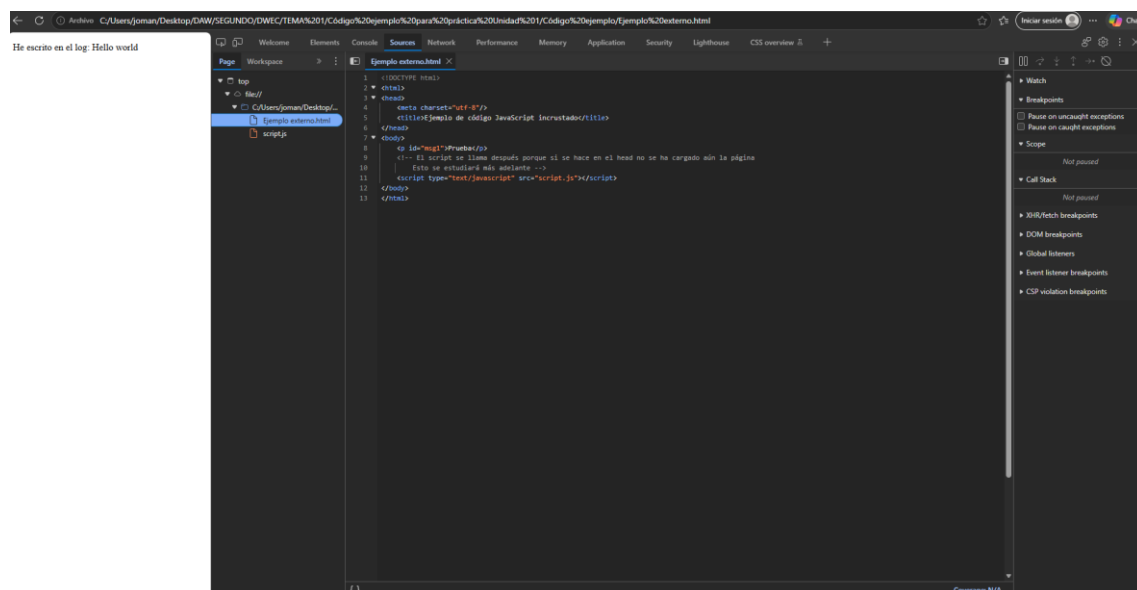
Microsoft Edge

Con Microsoft Edge son prácticamente los mismos pasos que hemos realizado con Google Chrome. Estos son los resultados:

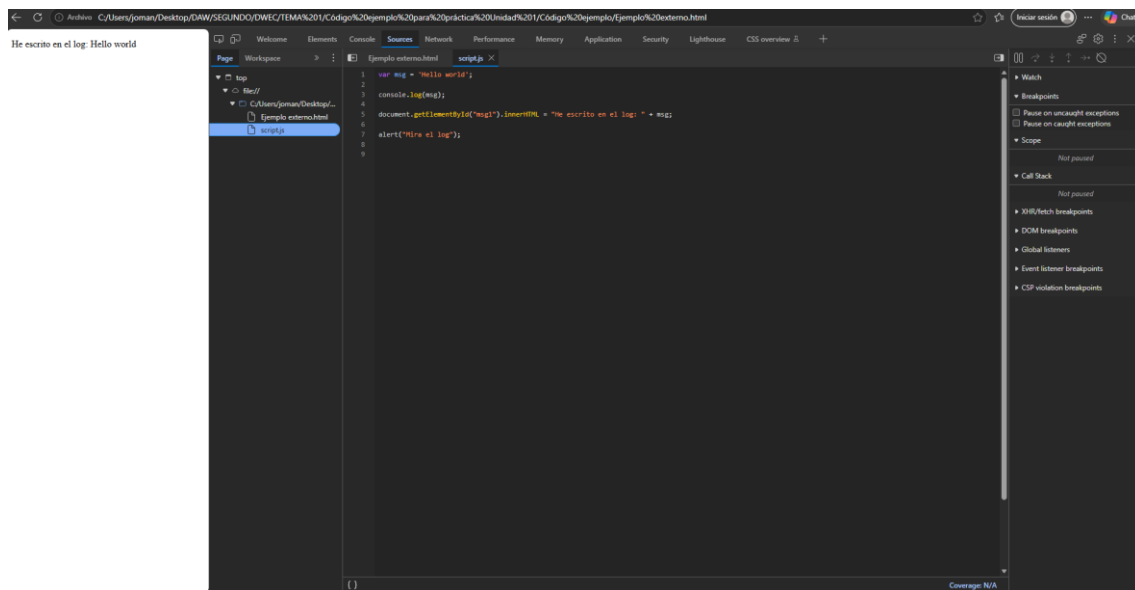
“Ejemplo incrustado.html”:



“Ejemplo externo.html”:



“script.js”

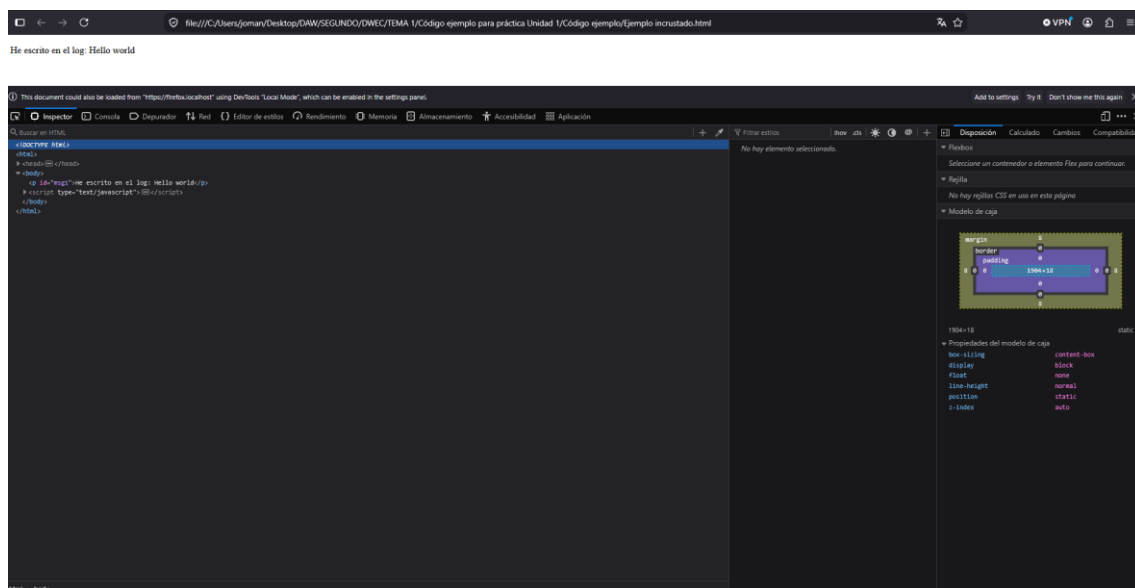


Mozilla Firefox

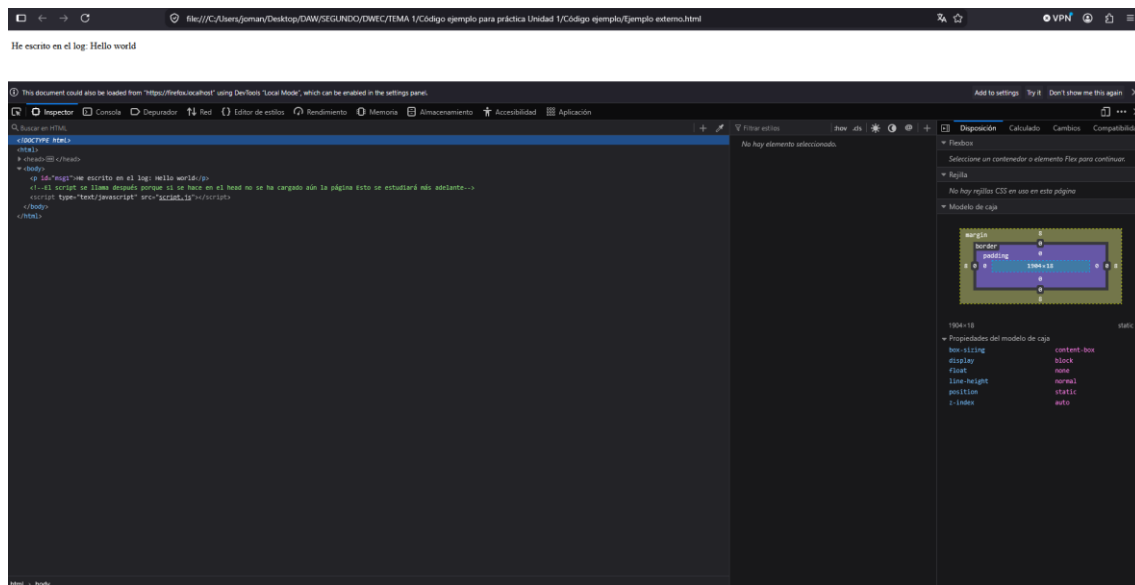
Con Firefox también es muy parecido, aunque “Sources” no existe como en Chrome y Edge.

Aquí utilizaremos “Inspector” para los archivos HTML y “Depurador” para los archivos JavaScript. Estos son los resultados:

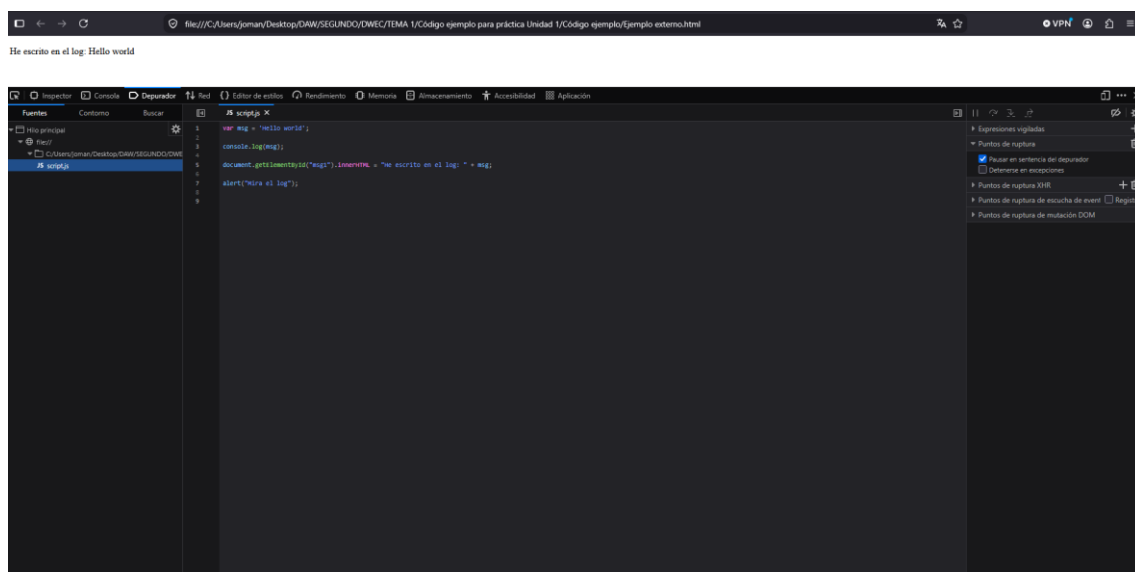
“Ejemplo incrustado.html”



“Ejemplo externo.html”:



“script.js”

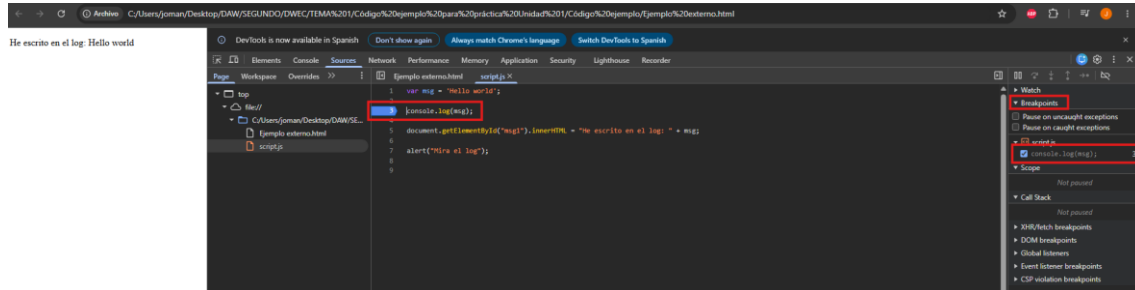


El siguiente punto que realizaremos es depurar el código estableciendo un punto de ruptura en “*console.log(msg)*”. Ejecutaremos el código en los tres navegadores depurando paso a paso desde esta sentencia.

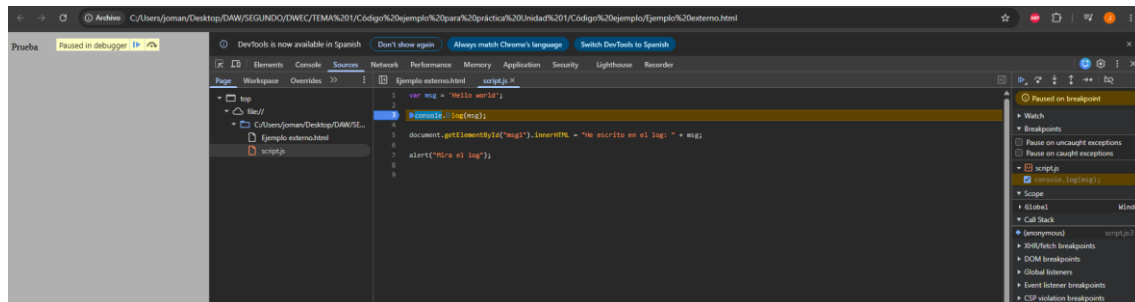
En este caso trabajaremos con “Ejemplo externo.html” porque así depuraremos también “script.js”.

Google Chrome

Para colocar el punto de ruptura o “*breakpoint*” volveremos a “*Sources*” dentro del *DevTools* y seleccionaremos “*script.js*”. En la línea de “*console.log(msg)*” haremos click izquierdo o con el click derecho seleccionaremos “*Add breakpoint*”. En el panel de la derecha podemos ver que lo hemos colocado en la línea 3:

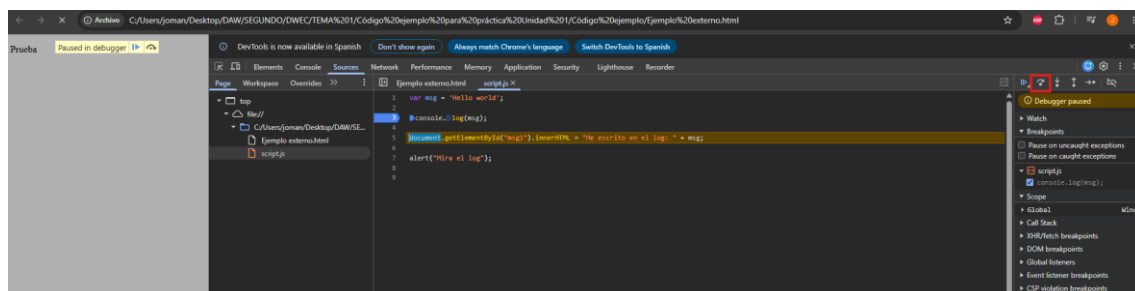


Después recargamos la página. Podemos ver que la línea queda resaltada y que Chrome indica que la ejecución está pausada:

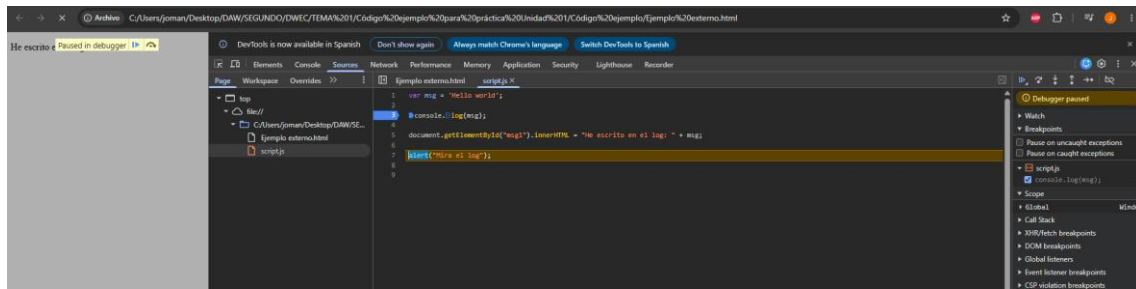


Ahora vamos a depurar paso a paso. Para ello pulsaremos “*F10*” o seleccionaremos “*Step over next function call*” (es el icono que tiene una flecha que pasa por encima del punto)

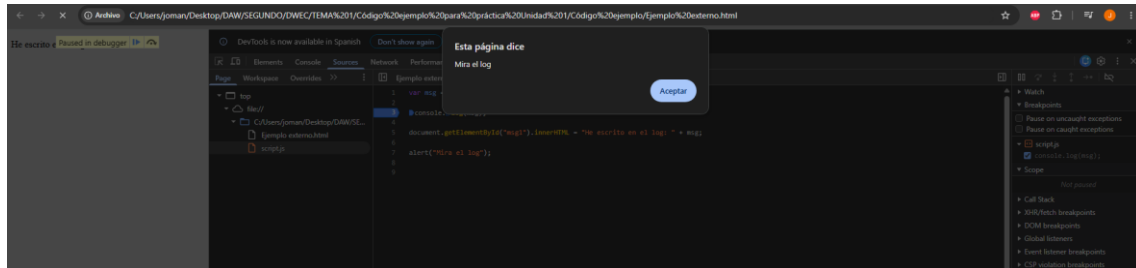
Step Over permite ejecutar la siguiente instrucción y luego detenerse. Cuando lo pulsemos, avanzará hasta la siguiente instrucción. Tal y como muestro en la siguiente captura Chrome ejecutará “*console.log(msg);*” y se quedará detenido en “*document.getElementById("msg1").innerHTML =*”:



Pulsamos de nuevo “*F10*”:



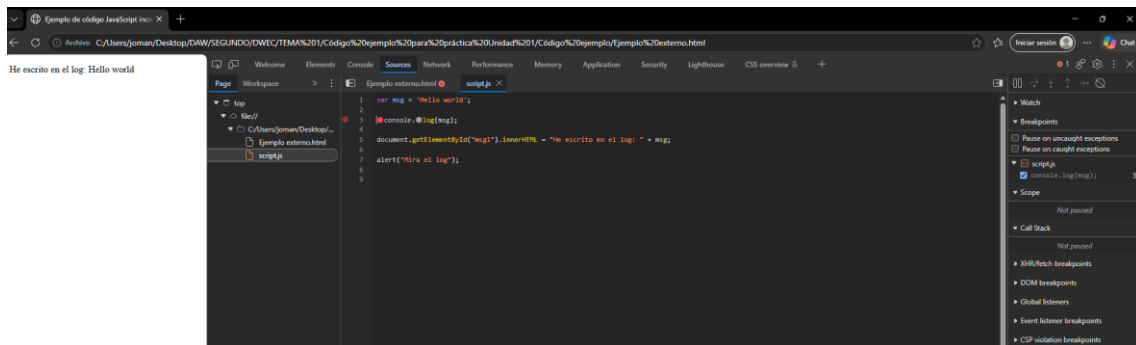
Y de nuevo. En este momento aparecerá la alerta:



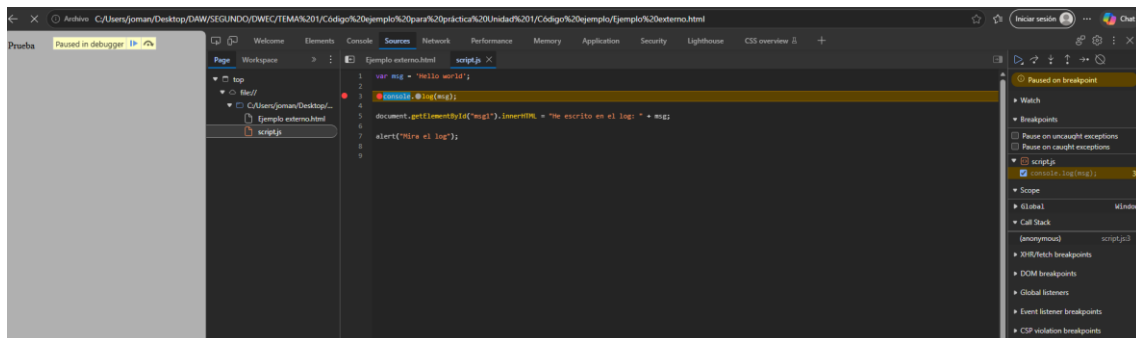
Microsoft Edge

Igual que antes, con Microsoft Edge son prácticamente los mismos pasos que hemos realizado con Google Chrome. Estos son los resultados:

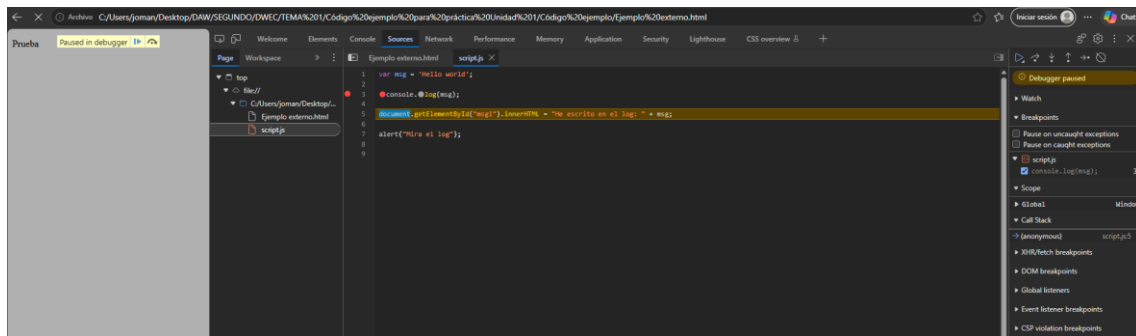
Marcamos el “breakpoint”:



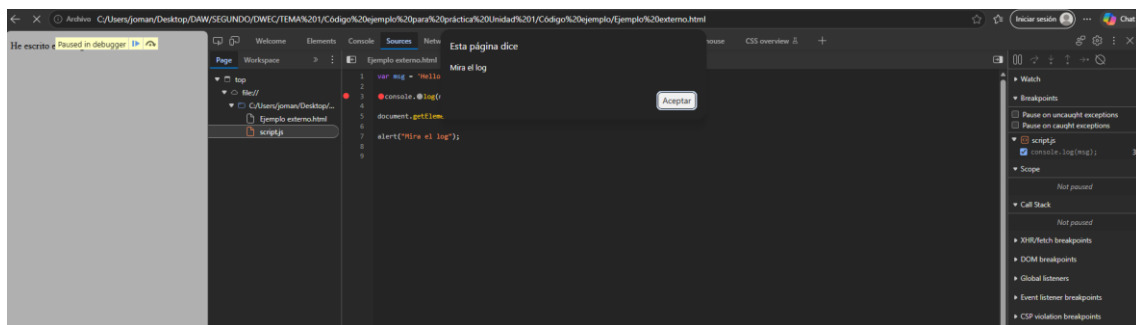
Recargamos la página. Nos indica que la ejecución está pausada y en qué línea:



Pulsamos “F10”:



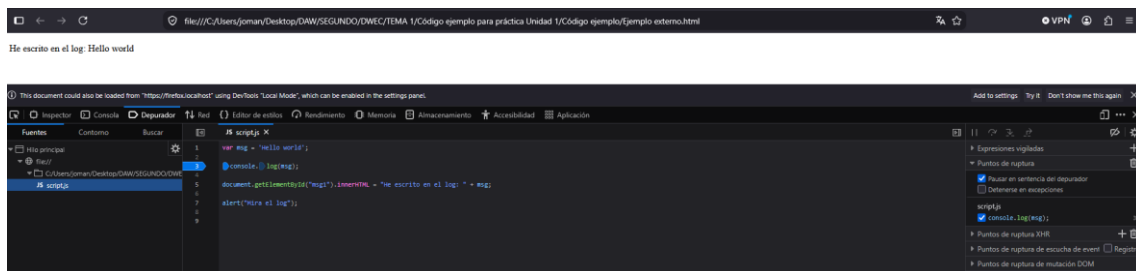
De nuevo “F10”. Aparecerá la alerta:



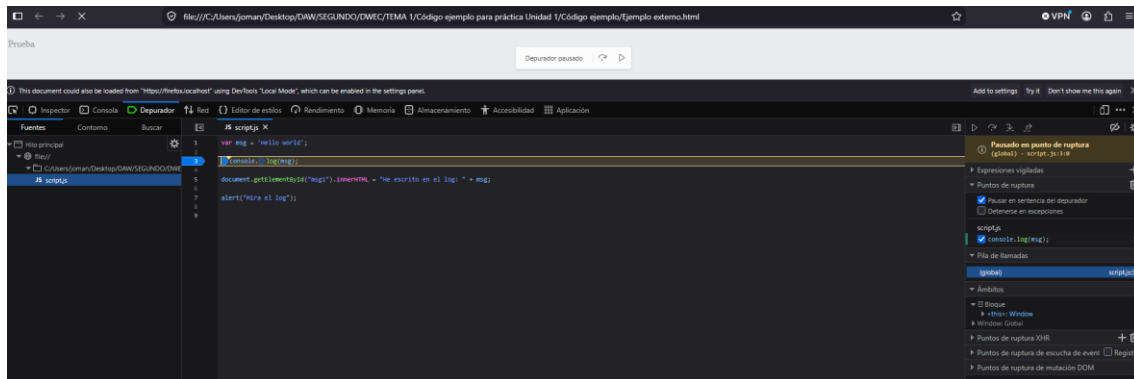
Mozilla Firefox

Con Firefox es prácticamente lo mismo. Estos son los resultados:

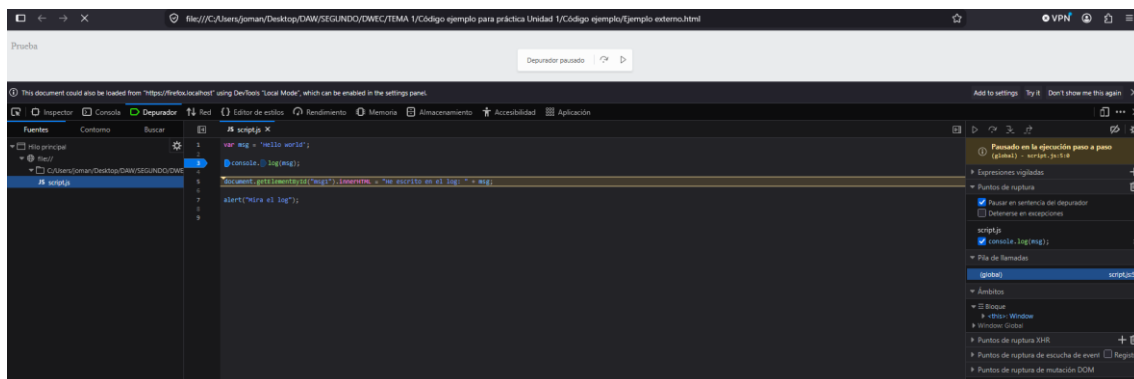
Marcamos el “breakpoint”:



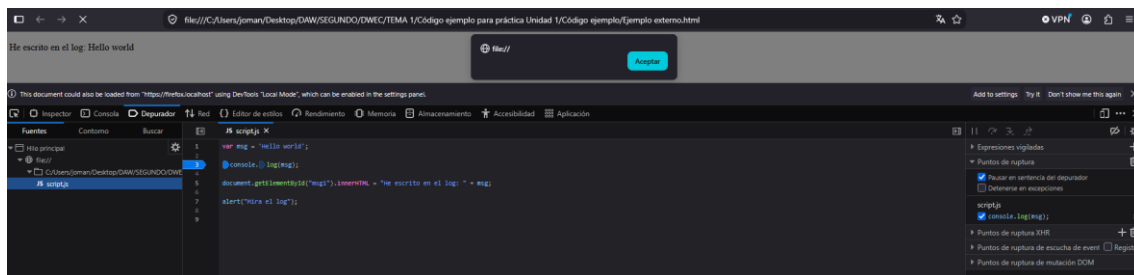
Recargamos la página. Nos indica que la ejecución está pausada y en qué línea:



Pulsamos *F10*:



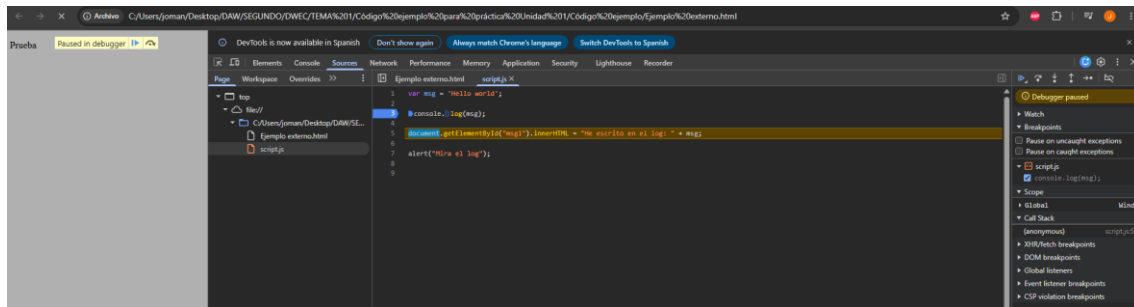
De nuevo “*F10*”. Aparecerá la alerta:



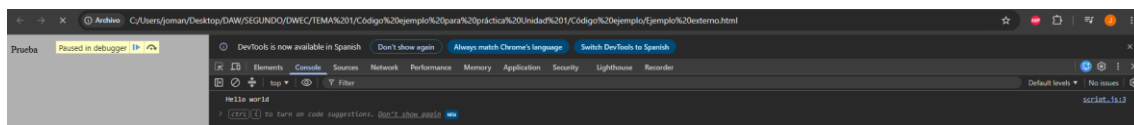
En este último punto buscaremos dónde está la consola en cada uno de los navegadores, para ver si aparece algún texto tras ejecutar el código.

Google Chrome

En Chrome debemos comprobar que el *breakpoint* estaba en la línea 3 y que hemos pulsado “*Step Over*” o “*F10*” para que avance a la siguiente línea, tal y como se puede ver en la siguiente captura:



Luego iremos a “Console” y debe aparecer “Hello World”:

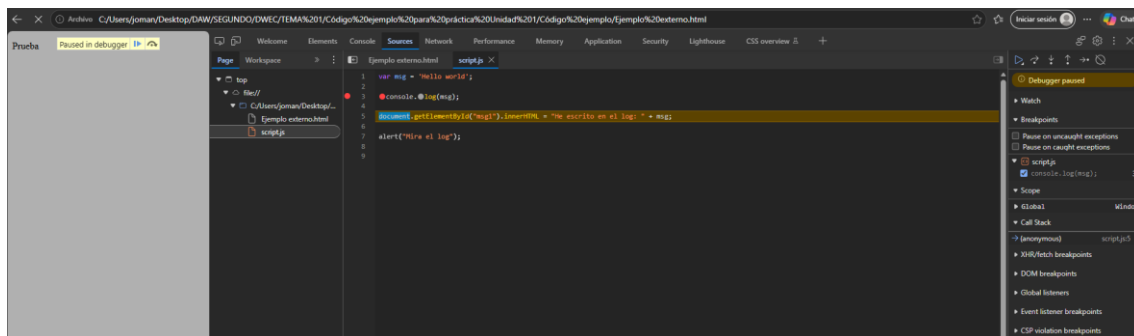


Esto demuestra que la línea “`console.log(msg)`” ha funcionado.

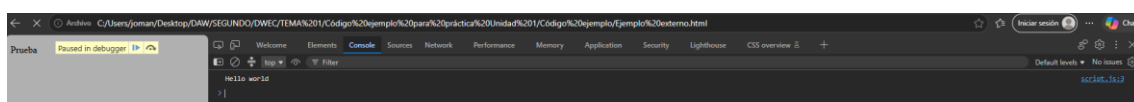
Microsoft Edge

Igual que antes, con Microsoft Edge son prácticamente los mismos pasos que hemos realizado con Google Chrome. Estos son los resultados:

Step Over en la línea 5:



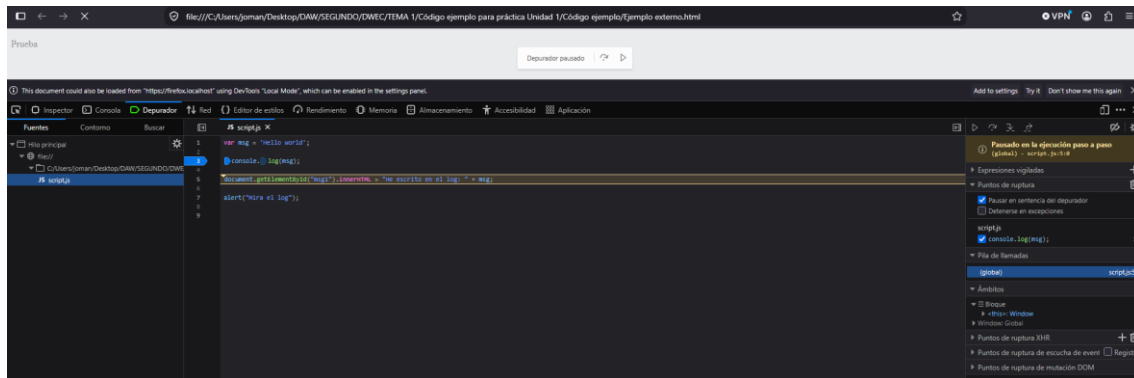
El mensaje “Hello World” en la consola:



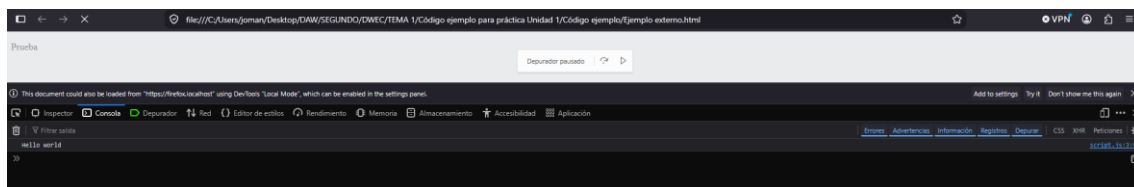
Mozilla Firefox

Con Firefox es prácticamente lo mismo. Estos son los resultados:

Step Over en la línea 5:



El mensaje “Hello World” en la consola:



****FIN DE LA PRÁCTICA – CONCLUSIÓN – RESUMEN****

En esta práctica hemos trabajado con diferentes herramientas para **desarrollar y ejecutar código JavaScript**. Hemos utilizado **Visual Studio Code, Notepad++ y JSFiddle**, comprobando las diferentes formas que tiene cada herramienta de ejecutar los archivos.

También hemos probado los ejemplos en **Google Chrome, Microsoft Edge y Mozilla Firefox**, utilizando sus **herramientas de desarrollador** para visualizar el código, acceder a la **consola** y **depurar los programas**. Además, hemos aprendido a utilizar **puntos de ruptura** y la **ejecución paso a paso** para comprobar cómo se ejecuta el código y observar el valor de las variables.

Finalmente, hemos comprobado que mediante **console.log()** podemos mostrar información en la **consola del navegador**, lo que resulta muy útil para **comprobar y detectar posibles errores** durante el desarrollo de una aplicación web.